

부트로더 u-boot #2 개발툴과 컴파일

글쓴이 : [고도리](#) (2004 년 07 월 02 일 오전 11:40) 읽은수: 8,638

[[임베디드강좌/윤덕배](#) ]

2 장

안녕하세요...고도리입니다.

오늘은 할일은 많은데, 정말 일하기가 싫어서 땡땡이 칠랍니다...^^
sound driver 랑 irq 루틴을 손봐야하는데 어떻게 할까 고민중이라...

하여간

이 부분은 옛날에 썼던걸 다시 정리해서 올리는 겁니다.
뭐 별루 어려운것은 없을겁니다.

일단 u-boot 를 분석하기에 앞서서요 환경설정이랑 간단하게 컴파일 하는법, 그리고
u-boot 에 기존에 있는 보드와 비슷한 custom board 를 만들었을때 새로운 보드이름을
만들고 그것에 알맞게 설정하는 간단한 방법등에 대해서 쓰겠습니다.

참, target 은 S3C2410 입니다. 물론 다른 arm 용에도 거의 해당되는 얘기니
큰 어려움은 없을겁니다.(S3C2400/S3C2440 등등)

이 칩은 PDA 용도로 만들어진것입니다. 스트롱암이나 Xscale 이랑 유사한데 가장 큰
장점은 nand controller 가 내장되어 있고, nand boot 가 가능하다는 겁니다.

속도 괜찮습니다...싼편이고요.

s3c2410 cpu 에 대한것은 삼성사이트에 가면 있습니다.

2 가지 버전이 있는데 제가 다루는 것은 S3C2410X(202MHz)입니다. S3C2410A 는 그냥
클럭만

266Mhz 로 빨라졌다고 생각해 버리시면 편합니다.

타겟보드는 smdk2410 으로 하고요...^^(물론 2410TK 보드도 상관없습니다.)

1. Cross Development Kit(즉, 간단하게 컴파일러)

부트로더로 Universal Boot(u-boot: <http://sourceforge.net/projects/u-boot>)를 사용합니다. 제일 다루기 쉽고, 구조적으로도 괜찮고요...

뭐니뭐니해도 여러가지 cpu 에 대한 부트로더를 그나마 일관성있게 다룰 수 있어서 좋아하는 편입니다. 개인적으로 가장 많이 아는 부트로더니까 선택하기도 한것이고요.

컴파일러로는 기존에 돌아다니는 arm 용 컴파일러를 써도 되고 직접 만들어서 해도 되지만 제가 추천하는 것은 u-boot 만든 사람들이 개발툴로 내놓은 ELDK(Embedded Linux Development Kit: 맞나?)

와 Montavista 에서 옛날에 공짜로 풀었던 HHL 2.0 JE 의 strongARM 용 컴파일러입니다.

참고로, 직접 만드시는것도 괜찮습니다(이것은 kelp 의 유영창님강좌에 있고, 그것을 가지고 수정한것을 <http://cafe.daum.net/eLinux> 에 토마스란 친구가 올린것도 있으니 참조하시고요)

1) 현재 arm 용으로는 arm920TDI 용이 나와있습니다.

사이트는 www.denx.de 이고요. 거기 가서 sw 쪽을 보면 ELDK 다운로드 받는 방법이 있습니다.

그것을 보고 리눅스에서 다운로드 받아서 거기 readme.html 파일에서 ISO 파일로 만들면 됩니다.

물론 인스톨방법도 쉽습니다. 그냥 install 명령어 시키는대로 해주면 됩니다.

2) HHL 2.0 JE 의 strongARM 용을 써도 됩니다.

Montavista 에서 옛날에 공짜로 풀었던 녀석인데 저는 공짜로 아주 잘쓰고 있습니다.

특히 app 컴파일엔 이게 최고입니다. 참고로 개발을 하다보면 nfs root fs 를 써야하는데 여기에 딸려있는 nfs root 가 정말 좋습니다.

설치법은 쉬우니깐....KESL 의 PPC 게시판에 보면 있을겁니다.

※ 참고로 nfs root fs 가 몬가?

리눅스나 유닉스 계열은 일단 3 박자가 맞아야 부팅이 제대로 됩니다.

- 부트로더
- 커널
- root filesystem

임베디드를 하다보면 부트로더, 커널은 그냥 구해다가 컴파일 하면 됩니다.
그러나 커널이 다 부팅이 되면 **init** 란 맨 상위 프로세스를 돌리게 되는데
이건 어디있을까용...^^, 즉, **root filesystem** 이란 녀는 OS 가 동작하는데 필요한
기본적인 파일들을 말하는 겁니다.

그래서 **ramdisk, nfs, mtd+jffs2** 등등...

임베디드의 장비의 경우 저장장소가 적기 때문에 **root fs** 를 넣으려면(이게 좀 큼니다)
꼭꼭눌러서 담거나, 아니면 네트워크 디바이스가 있을경우 윈도우즈의 공유개념으로
다른데 넣어놓고 그것을 활용하는 방법을 씁니다.

꼭 눌러담는 방법이, 압축해서 쓰는 **ramdisk** 나 **mtd+jffs2** 등이고요,
공유방법(전통적인 유닉스 계열에서는 **nfs** 라고 하죠...ㅎㅎ)중 하나가 **nfs root fs** 라는
겁니다.

즉, 다른데다 자기의 **root fs** 를 넣어놓고 그냥 원격으로 로딩해서 쓰는거죠.

이것의 좋은점은 주로 **nfs root fs** 를 자기 개발 **host** 에 넣어 놓으면 **app** 개발시 꼭 장비로
다운로드 안받고 **nfs root fs** 디렉토리로 복사하면 장비에서 바로 사용할 수가 있어서
개발이 편합니다.

2. 부트로더 컴파일 해보기

여기서는 먼저 **custom board** 용이 아닌 **s3c2410** 용 **ev board** 인 **smdk2410** 용으로
컴파일을 해보는 것입니다.

일단 받아오세요....어디가서...앞에서 설명한

<http://sourceforge.net/projects/u-boot> 에서(버전은 큰 상관이 없습니다. 그냥
암거나...)

압축을 풉니다.(윈도우에서 풀지마세요...^^)

bzip2 -d u-boot....(어찌구저찌구)

그 다음에 tar 파일을 풉니다.

tar xf u-boot.....

여기서은 파일이름을 정확하게 몰라서 쓴거니 정말으로 따라하면 안됩니다.(가끔 하지 말래면 하는사람이 있습니다...^^)

u-boot 가 풀린 디렉토리로 들어갑니다.

Makefile 에서 다음과 같은 부분 수정

1) cross compiler prefix 추가

크로스 컴파일러를 어느 cpu 용으로 설정할까 결정해주는 부분입니다.
vi 로 Makefile 을 편집합니다.(설마 vi 를 못쓴다고는 하지 않겠죠?)

vi Makefile

한 52 번째줄 쯤에 이런 부분이 있습니다.

```
...
ifndef CROSS_COMPILE
ifeq ($(HOSTARCH),ppc)
CROSS_COMPILE =
else
ifeq ($(ARCH),ppc)
CROSS_COMPILE = ppc_8xx-
endif
ifeq ($(ARCH),arm)
CROSS_COMPILE = arm-linux-
```

```

endif
ifeq ($(ARCH),i386)
ifeq ($(HOSTARCH),i386)
CROSS_COMPILE =
else
CROSS_COMPILE = i386-linux-
endif
endif
ifeq ($(ARCH),mips)
CROSS_COMPILE = mips_4KC-
endif
endif
endif

```

```
export CROSS_COMPILE
```

```
...
```

요 부분앞에

```
CROSS_COMPILE = arm_920TDI-
```

이런 녀를 넣어줍니다.

(여기서 저는 **ELDK** 를 쓰기 때문에 이렇게 한 것이고 다른 종류의 컴파일러를 쓸 경우는 **prefix** 를 다르게 바꿔줘야 합니다. 예를들어 **HHL 2.0** 의 것을 쓴다면 **arm_sa_le-**이고 그냥 **arm-linux-gcc** 를 쓴다면 **arm-linux-** 입니다.)

그래서 결과적으로는 담과 같이 되겠죠

```
...
```

```
CROSS_COMPILE = arm_920TDI-
```

```

ifndef CROSS_COMPILE
ifeq ($(HOSTARCH),ppc)
CROSS_COMPILE =
else

```

```

ifeq ($(ARCH),ppc)
CROSS_COMPILE = ppc_8xx-
endif
ifeq ($(ARCH),arm)
CROSS_COMPILE = arm-linux-
endif
ifeq ($(ARCH),i386)
ifeq ($(HOSTARCH),i386)
CROSS_COMPILE =
else
CROSS_COMPILE = i386-linux-
endif
endif
ifeq ($(ARCH),mips)
CROSS_COMPILE = mips_4KC-
endif
endif
endif

export CROSS_COMPILE
...

```

2) examples 를 compile 하지 않게 만드는 부분

※ 이부분은 안해도 될 수 있으니깐 이부분 건너뛰고 일단 컴파일 함 해보고 안되면 이걸 해보세요

example 컴파일하다가 에러가 나는경우가 있으니깐 에러가 날 경우는 빼버리세요.

```

SUBDIRS = tools \
lib_generic \
lib_$(ARCH) \
cpu/$(CPU) \
board/$(BOARDDIR) \
common \

```

```
disk \
fs \
net \
rtc \
dtb \
drivers \
post \
post/cpu \
examples
위의 부분을 이렇게
SUBDIRS = tools \
lib_generic \
lib_$(ARCH) \
cpu/$(CPU) \
board/$(BOARDDIR) \
common \
disk \
fs \
net \
rtc \
dtb \
drivers \
post \
post/cpu
```

바꿔준다.

3) make 를 한다.

자 이제는 자기 보드에 알맞는 **configuration** 설정을 해주고 부트로더를 컴파일 할 때입니다.

여기서는 그냥 컴파일만 해보는 것이니깐 s3c2410 의 ev 보드인 smdk2410 보드를 타겟으로 합니다.

```
shell> make clobber
```

```
shell> make smdk2410_config
```

```
shell> make
```

요렇게 하면 **u-boot.bin** 이란 파일이 생성되어 있을 겁니다.

make clobber 는 **make clean** 과 같은겁니다. 새로운 보드에 대해서 컴파일 하기전에 이전것들에대한 설정을 싸그리 지우는 겁니다.

make smdk2410_config 는 특정보드에 대한 **configuration** 을 하는 겁니다.
실제로는 **smdk2410.h** 를 **config.h** 로 바꾸는 것입니다....여기서 결과적으로는 **cpu** 는 **arm920t** 로 **board** 는 **smdk2410** 으로 지정이 됩니다.

smdk2410 보드를 타겟으로 했기때문에 이렇게 환경설정을 한겁니다.

그다음은 **make** 하는 겁니다...

여기서 만들어진 **u-boot.bin** 을 **jtag** 혹은 **ROM-writer** 등을 이용해서 **flash** 에 **write** 하면 부팅이 됩니다.

3. custom board 용 설정등을 만들고 컴파일 하기

여기서는 개발하는 혹은 개발해둔...등등...

custom board 용 개발환경을 새로 만드는 것입니다.

smdk2410 에 대한 것을 가지고 손대는 것도 괜찮겠지만...기왕이면 자기 디렉토리를 하나 만들어서 하는게 낫겠지요?

여기서 **custom board** 를 뭐라고 할까요...저는 **godori2410** 으로 했습니다.

머리가 창조성이 결핍된 녀미라서 이정도가 한계입니다....^^

참, 여기서 쓰는 내용은 **ppc** 등의 경우에는 약간씩 틀립니다. 이건 환경에 맞춰야 하는 부분이라서요...지금 설명하는 것은 **ARM** 에 국한된겁니다.

자 그럼 시작하죠...

u-boot 소스 디렉토리로 갑니다.

1) board/smdk2410 를 board/godori2410 로 복사한다.

일단 자기보드 디렉토리를 만드는 것인데, 가장 비슷한 보드중에서 복사를 하는것이 좋습니다. 아니면 직접 만들어도 되는데 그건 알아서 하세요...
제가 추천하는 것은 "베끼세요..." ^^

2) include/configs/smdk2410.h 를 include/configs/godori2410.h 로 복사한다.

이것도 1)과 마찬가지로입니다. 직접 작성해도 되나 힘이 듭니다.

3) Makefile 에 자기 보드 configuration 을 잡는 부분을 추가한다.

u-boot 에 있는 top 디렉토리의 Makefile 을 수정보면 되는데 일단 vi 로 open 합니다.

vi Makefile

smdk2410 라는 단어로 검색을 하면 다음과 같은데서 걸립니다.

```
smdk2410_config : unconfig
```

```
@./mkconfig $(@:_config=) arm arm920t smdk2410
```

요 두줄을 복사해서 꼭 같은걸 하나 더 만듭니다. 밑에다가 다음과 같은 두줄을 만듭니다.

```
godori2410_config : unconfig
```

```
@./mkconfig $(@:_config=) arm arm920t godori2410
```

그리고 저장....

5) 원래는 include/configs/smdk2410.h 파일 윗쪽에 선언된

```
#define CONFIG_SMDK2410 1 /* on a SAMSUNG SMDK2410 Board */
```

라고 되어 있는 놈도 내 보드에 알맞게 바꿔줘야하는데 귀찮으니깐 넘어가죠...^^
(복사를 했으니 godori2410.h 이겠죠)

특별하게 이 부분은 바꿔주지 않아도 괜찮습니다...꼭 바꿔야 하겠다 그러면 바꿔도 괜찮습니다.

※ 만일 **ppc** 의 경우면 말이 좀 틀려집니다. **s3c2410** 의 경우 **ev board** 랑 굉장히 유사하게 보드를 만들고, 또 **ev** 보드 자체가 실무에서도 쓰기 좋게 되어 있기 때문에 그냥 써도 된다는겁니다. **ppc** 는 워낙 보드가 변종이 많아서리....^^

6) board 디렉토리의 **Makefile** 을 수정한다.

board/godori2410 디렉토리로 가서 **Makefile** 을 열어서 **smdk2410** 이라고 되어 있는 놈을 **godori2410** 으로 바꾼다.

7) 필요할 경우 해당보드의 **Linker script** 파일을 수정본다.

s3c2410 의 경우는 손볼일이 없었습니다. **ppc** 는 손보는 경우가 있습니다.

board/godori2410/u-boot.lds 를 열어서 보고 수정볼데 있음 수정보고 아니면 넘어갑니다...(주로 넘어가겠죠?)

8)자 이제 컴파일을 한다. 어디서 당근 **u-boot top** 디렉토리지

make clobber

make godori2410_config

make

여기까지 일단락 하고요. 오늘 아예 놀 생각이니...^^, 시간나면 설정파일인 **smdk2410.h** 를 분석하도록 하지요.

그 분석이 끝나면 바로 스타트업 코드로 들어가던가 하겠습니다.